



Buglt
by Taskivator

Agent QA de création de bugs pour VS Code

Buglt

Transformez des notes de test sommaires en tickets de bug soignés et vérifiés, créés dans votre outil de suivi en quelques secondes.

Un agent QA qui s'adapte au workflow de votre équipe.

Conçu par un ingénieur QA senior qui rédige et relit des rapports de bug chaque jour.

 Privé par conception : votre travail n'est jamais envoyé à Taskivator.

~10 min économisées par bug	Anti-doublons avant la création	~15 min configuration unique	\$39.99 Solo lancement · achat unique · puis \$59.99
---------------------------------------	---	--	--

Guide de démarrage · À lire une fois, à configurer en ~15 minutes, pour créer des tickets impeccables.

CONFIANCE

Sécurité et confidentialité intégrées

Ce ne sont pas des options : elles sont actives par défaut, et c'est pour ça que les équipes font confiance à BugIt avec de vrais tickets.

Nous ne voyons jamais votre travail

BugIt s'exécute dans votre éditeur ; vos tickets ne sont transmis qu'au modèle d'IA et à l'outil de suivi que vous connectez. La seule chose qui nous parvient, ce sont des données de licence/mise à jour : la connexion à votre compte BugIt (dans le navigateur, sur le Portal), un identifiant d'appareil haché à sens unique, un identifiant d'installation aléatoire, le nom de votre appareil (nom d'hôte) et le nom de votre système d'exploitation, la version de BugIt et le filtre d'offre que vous choisissez, du matériel cryptographique de défi à durée de vie courte, un secret d'accusé de réception par activation (son empreinte, et la valeur elle-même uniquement si vous révoquez l'accès de cet appareil), et le droit Solo ou Team que vous approuvez pour cet appareil. Il n'y a aucun code de licence, et votre mot de passe reste dans le navigateur et n'est jamais saisi dans VS Code. Aucune télémétrie, aucune analyse, jamais.

Sauvegardes automatiques

Avant chaque mise à jour (et à tout moment sur demande), votre configuration, votre glossaire, votre style maison, vos endpoints MCP et vos préférences apprises sont capturés sur votre appareil (votre droit d'appareil signé et les valeurs des identifiants sont exclus). Une seule ligne les restaure : `Restore my settings`.

Rien n'est créé sans votre confirmation

Chaque ticket et commentaire est présenté en aperçu ; les actions irréversibles exigent de saisir `FILE IT`, car un simple « oui » ne suffit pas. Dites `dry run` pour un mode d'entraînement : tout ticket, commentaire, modification, pièce jointe, suppression et notification externes sont bloqués (les commandes locales que vous lancez vous-même, comme la sauvegarde ou l'export, peuvent encore créer des fichiers locaux) et l'agent refuse les écritures dans l'outil de suivi ; ce refus suit les instructions de BugIt, ce n'est pas un verrou de plateforme, utilisez des identifiants en lecture seule pour évaluer.

Mises à jour signées et vérifiées

Les mises à jour sont signées cryptographiquement avec une clé conservée *séparément* de la licence. L'agent vérifie chaque octet et refuse tout élément altéré, puis conserve tous vos paramètres.

Aucun secret lisible sur le disque

Les jetons rejoignent le magasin d'identifiants de votre système et ne sont jamais lisibles sur le disque : sous Windows sous la forme d'un enregistrement chiffré avec la clé de votre propre compte Windows, et sous macOS et Linux le secret lui-même réside dans le Trousseau ou le service libsecret. `config.json` ne contient que des organisations et des URL, et un validateur signale tout ce qui ressemble à un secret divulgué avant même son enregistrement, que ce soit dans la config ou dans une commande exécutée par l'agent pour vous.

Résistant à l'injection de prompt

Le texte extrait de pages ou de tickets est traité comme une donnée, jamais comme une commande : les instructions injectées sont signalées, jamais exécutées, et portées à votre attention.

Personnalisez en toute sécurité

`config.json`, le style maison, le glossaire et la configuration MCP sont à vous, conservés à chaque mise à jour. Le fichier de l'agent et les autres fichiers de `.github/instructions/` ne le sont pas : les modifier à la main les fait écraser silencieusement, sans sauvegarde permettant de les restaurer.

Privé par conception. Vos rapports de bug, vos specs, votre glossaire et vos paramètres restent les vôtres. Votre travail ne va qu'aux outils que vous avez connectés et à votre propre modèle d'IA, jamais à nous. Les données de licence et de mise à jour décrites ci-dessus sont la seule chose que BugIt envoie.

Avertissement sur la traduction.

Ce document a été traduit automatiquement et n'a pas été relu par des locuteurs natifs. La version anglaise fait foi : en cas de divergence, le texte anglais prévaut. Pour la formulation la plus exacte et la plus à jour, veuillez vous reporter au document en anglais.

POUR COMMENCER

Glossaire en 60 secondes

Aucune compétence technique requise. Voici les seuls mots qui comptent, en termes simples.

VS Code

Une application gratuite de Microsoft, la « fenêtre » dans laquelle vit cet agent.

GitHub Copilot

Le cerveau de l'IA qui rédige les rapports. Coût total : la licence BugIt à achat unique plus un abonnement Copilot (ou l'usage de votre propre fournisseur d'IA).

BugIt

Votre assistant dans cette fenêtre. Discutez avec lui comme avec un collègue.

Outil de suivi

Là où votre équipe conserve ses bugs (Jira, ADO, GitHub...).

Jeton d'API

Un mot de passe privé que vous créez dans votre propre compte d'outil de suivi, pour que BugIt crée les tickets en votre nom. Il est conservé dans le magasin d'identifiants de votre système d'exploitation : chiffré avec la clé de votre propre compte sous Windows, et dans le Trousseau ou libsecret sous macOS et Linux. Copier le dossier sur une autre machine ne copie rien d'exploitable.

Serveur MCP

Un pont facultatif pour lire du contexte supplémentaire (Confluence, Sentry, Notion). La création de tickets n'en a pas besoin.

config.json

Une fiche de paramètres. L'agent la remplit pour vous.

Chat

Là où vous tapez : « Write a bug report: l'appli plante à l'enregistrement ».

Si vous savez installer une application, vous connecter et taper une phrase, vous savez utiliser cet outil.

COÛT ET LICENCE

Combien ça coûte

Compatible avec votre modèle d'IA préféré. BugIt pilote le modèle à l'intérieur de GitHub Copilot (GPT, Claude ou Gemini), ou fonctionne en mode autonome avec votre propre clé.

Gratuit

L'offre Copilot Free couvre un usage occasionnel.

Aucune clé API

Aucun hébergement, aucun frais par bug.

Copilot Pro

Le palier payant de GitHub pour un usage quotidien.

Pas de Copilot ?

Apportez votre propre clé OpenAI/Anthropic.

\$39.99

prix de lancement, puis \$59.99 : un achat unique pour une durée d'un an, un appareil à la fois, toutes les fonctionnalités. **Team est désormais disponible : \$199 (puis \$249.99), un achat unique pour 5 postes de membre**, gérés de façon centralisée dans le BugIt Portal. Plus de cinq personnes ? Ajoutez une deuxième licence Équipe à la même Équipe et les postes s'additionnent.

C'est un **paiement unique pour une année entière**, sans facturation mensuelle et sans reconduction automatique. Les outils QA concurrents facturent leur prix **tous les mois** ; économisez le temps d'une **poignée de bugs** et c'est déjà rentabilisé.

Licence Équipe

Une seule commande partage votre configuration d'outil de suivi, votre glossaire et votre style de ticket avec toute l'Équipe (`share.py export` , les coéquipiers font `import`), pour que tout le monde crée des tickets exactement au même format dès le premier jour. Dans le BugIt Portal, vous invitez des personnes par e-mail, vous leur donnez le rôle Administrateur ou Membre, vous voyez qui a activé un appareil et vous désignez quelqu'un qui pourra reprendre l'Équipe si vous êtes injoignable. Chaque membre se connecte avec son propre compte et utilise son propre appareil : ni clé partagée, ni connexion partagée.

POURQUOI ÇA EN VAUT LA PEINE

Ce que vous obtenez réellement

Pas « une IA qui rédige des bugs », mais un agent QA qui s'intègre à la façon dont votre équipe travaille déjà.

SANS BUGIT	AVEC BUGIT
Rédiger chaque bug à la main	Une phrase → un ticket complet et vérifié
Les doublons s'accumulent sur le tableau	Anti-doublons automatique avant la création, même en cas de formulation différente
Le format varie selon chaque testeur	Un modèle cohérent, à chaque fois
Traduire et consulter les specs à la main	Multilingue, et le contexte est récupéré pour vous

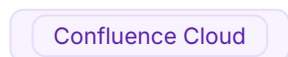
Compatible avec l'outil de suivi que votre équipe utilise déjà

Buglt rédige le ticket pour n'importe quel outil de suivi et le crée via le connecteur que vous activez. Changez à tout moment.

Crée les tickets avec un identifiant que vous créez dans votre propre compte, que Buglt vérifie face à votre destination avant de l'enregistrer :



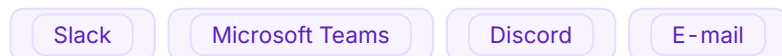
Source de connaissances assistée (Atlassian RoVo MCP) :



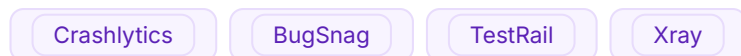
Expérimental, un point de terminaison connu que Buglt configure et vérifie en direct sur votre compte :



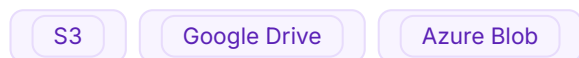
Prévient votre équipe qu'un ticket a été créé, intégré : Slack demande un jeton de bot et un canal, Microsoft Teams une URL Workflows, Discord une URL de webhook et l'e-mail vos paramètres SMTP. Buglt envoie un vrai message de test avant d'enregistrer le réglage.



Accompagnement à la configuration uniquement, via votre propre serveur MCP compatible :



Non pris en charge par la configuration automatisée :



Il vérifie aussi les doublons, recherche les crashes, récupère le contexte des specs, traduit entre langues, et rédige les commentaires de vérification/clôture.

CONFIGURATION

Opérationnel en 6 étapes

Configuration unique dans VS Code. La configuration vérifie ce dont vous avez besoin et vous demande votre accord avant toute installation. Buglt ne nécessite aucun paquet Python tiers.

1 Installez VS Code

Téléchargez-le depuis code.visualstudio.com et ouvrez-le.

2 Installez GitHub Copilot

Extensions → recherchez « GitHub Copilot » → Install → connectez-vous.

3 Ouvrez ce dossier

File → Open Folder. L'agent se charge depuis `.github/agents/`.

4 Sélectionnez l'agent

Dans le chat, basculez le menu déroulant de mode vers **BugIt QA Agent**.

5 Tapez « Begin setup »

Répondez à quelques questions ; l'agent écrit `config.json`. En cas d'interruption, le retaper reprend là où c'était plutôt que de tout recommencer.

6 Activez, puis connectez votre outil de suivi

L'activation se déroule dans votre navigateur : saisissez `Activate`, connectez-vous au BugIt Portal, puis approuvez cet appareil. Aucun code à coller. Exécutez ensuite une fois `python tools/connect.py jira` pour enregistrer votre propre jeton d'API.

Raccourcis : `preset.py game` pour partir d'un modèle · `setup_wizard.py` sans Copilot · `doctor.py` pour une liste de vérification de l'état de préparation.

Un manuel complet, étape par étape, est inclus.

Après votre achat, vous recevrez un guide de configuration et d'utilisation détaillé, clic par clic (en français), écrit pour des personnes qui n'ont jamais fait ça, afin que la configuration soit simple et que vous ne restiez jamais bloqué.

AVANT D'ACHETER : CE QU'IL VOUS FAUT

PRÉREQUIS	POURQUOI
VS Code (gratuit)	L'agent s'exécute à l'intérieur
GitHub Copilot (recommandé)	Alimente l'agent, ou apportez votre propre clé OpenAI/Anthropic
Python 3.10-3.14	Exécute les outils utilitaires
Un compte d'outil de suivi	Où vont les bogues. BugIt crée des tickets dans onze outils : Jira Cloud, Azure DevOps, GitHub Issues, GitLab Issues, Bugzilla, YouTrack, Linear, Shortcut, ClickUp, Asana et Trello. Chacun a une configuration guidée, chacun écrit avec un identifiant que vous créez dans votre propre compte, et BugIt vérifie cet identifiant face à votre destination avant de l'enregistrer

Bon à savoir

BugIt fonctionne dans **VS Code avec GitHub Copilot**, dans l'**extension Claude** et dans un terminal ordinaire ; il pilote **votre** modèle d'IA au lieu d'en fournir un ; il crée des tickets dans les **onze** outils qu'il nomme, chacun avec un identifiant que vous créez dans votre propre compte, même si ce qui atteint un champ de priorité triable varie selon l'outil et que trois d'entre eux n'en ont aucun ; Confluence, Sentry et Notion sont en lecture seule, et sont les seules à en avoir besoin ; les notifications Slack, Teams, Discord et e-mail sont intégrées ; et le caviardage et les autres garde-fous sont des aides, pas des garanties : **vous relisez et approuvez chaque ticket**, et les résultats sont plus réguliers avec un modèle d'IA plus puissant qu'avec un modèle très léger ou gratuit.

EN ACTION

Une phrase en entrée. Un ticket propre en sortie.

C'est tout l'intérêt de l'outil. Tapez-la comme vous la diriez à un collègue ; l'agent fait le reste.

VOUS TAPEZ

"the app crashes every time I tap Save on iPhone"



BUGIT RÉDIGE LE BROUILLON : VOUS RELISEZ, PUIS VOUS CONFIRMEZ EN SAISISANT « FILE IT »

[Crash] L'appli se ferme en appuyant sur Enregistrer (iPhone)

Étapes	1. Ouvrir l'appli 2. Effectuer une modification 3. Appuyer sur Enregistrer
Attendu	La modification est enregistrée et l'appli reste ouverte
Constaté	L'appli se ferme instantanément, à chaque fois
Détails	Reproductibilité 5/5 · Gravité Critique · Plateforme iOS (iPhone)

Puis il vérifie les doublons et crée le ticket dans votre outil de suivi après que vous avez confirmé en saisissant **FILE IT** (les actions irréversibles exigent **FILE IT**, car un simple « oui » ne suffit pas), en définissant le champ de priorité propre à l'outil de suivi lorsqu'il en possède un, et en inscrivant la gravité dans le ticket lorsqu'il n'en a pas, et vous transmet le lien. Envie d'ajuster avant ? Dites simplement « *make severity high* ». Envie de vous entraîner ? Dites **dry run** : il rédige tout sans écrire dans votre outil de suivi (ce refus suit les instructions de BugIt, ce n'est pas un verrou de plateforme ; utilisez des identifiants en lecture seule pour évaluer).

AU QUOTIDIEN, VOUS TAPEZ SIMPLEMENT

VOUS TAPEZ

Write a bug report: ...

VOUS OBTENEZ

Rapport complet, vérification des doublons, détection automatique de version

VOUS TAPEZ

VOUS OBTENEZ

Close #ID · Translate to ja: ...

Commentaire de vérification/clôture · traduction cohérente avec le glossaire

Update · Back up my settings · help

Mise à jour signée · instantané local · liste complète des commandes

EXTRAS FACULTATIFS

Les fonctionnalités avancées expliquées

Activez-les dans [Begin setup](#), puis déclenchez-les depuis le chat.

Triage digest

Tapez [Triage digest](#) pour un résumé de standup instantané : ce qui a été créé/modifié, le décompte par gravité et par catégorie.

Alertes SLA

Les bugs de gravité élevée reçoivent un marqueur SLA/échéance dès leur création, pour que les problèmes urgents ressortent automatiquement.

Export bug to file

[Export bug to file](#) enregistre le rapport en Markdown/JSON local au lieu de le créer. Idéal hors ligne ou pour garder une copie.

Champs par catégorie

Chaque catégorie ajoute ses propres champs (crash → pile d'appels, UI → capture d'écran) pour que chaque rapport capture le bon niveau de détail.

Undo last filing

[Undo last filing](#) affiche la dernière écriture du journal d'audit de BugIt et explique ce qui peut être récupéré. BugIt ne peut ni supprimer ni clôturer un ticket, et n'invente jamais de note de rétractation. Après un nouvel aperçu et [FILE IT](#), il peut seulement retirer un commentaire ou une pièce jointe qu'il a lui-même créés.

Tout est facultatif. Ignorez ces fonctionnalités et la boucle principale rédiger → relire → créer fonctionne parfaitement.

Une licence qui vous respecte : après une activation en ligne réussie, une licence en cache vous permet de travailler hors ligne jusqu'à 72 heures, après quoi une nouvelle vérification en ligne a lieu. Un appareil à la fois. Lorsque vous passez à un nouvel appareil, l'ancien ne peut plus renouveler immédiatement, et sa place se libère dès qu'il se reconnecte ou que sa licence de 72 heures expire.

Dépannage

SYMPTÔME	SOLUTION
« No tracker enabled »	Lancez <code>Begin setup</code> , choisissez un outil de suivi
La création de ticket est refusée	Exécutez <code>python tools/connect.py jira --check</code> et lisez les <code>fixes</code> qu'il énumère
Demande une réactivation	Saisissez <code>Activate</code> et terminez la connexion dans le navigateur. Buglt n'affiche jamais les valeurs des identifiants enregistrés.
Refuse de créer le ticket	Vérifiez <code>validate_config.py --test</code> ; confirmez l'outil de suivi principal + la connexion
Semble spécifique aux jeux	Modifiez <code>config.categories</code> selon votre domaine

Toujours bloqué ? Rendez-vous sur **bugit.dev** ou écrivez à **support@bugit.dev** (support assuré en anglais) : une vraie personne vous répondra. Souci de configuration dans vos 7 premiers jours ? Nous vous aiderons à démarrer, ou à obtenir un remboursement via la boutique.

Une année entière, en un seul paiement

\$39.99, c'est un paiement unique pour une année entière, et le compteur démarre le jour de l'achat. Il n'y a aucun frais par bug et aucune facturation mensuelle. Les outils QA concurrents facturent leur prix **tous les mois**. Économisez le temps d'une poignée de bugs et c'est déjà rentabilisé.

~10 min économisées/bug

une ligne → un ticket complet

Anti-doublons

avant la création

Compatible avec votre outil de suivi

celui que vous utilisez déjà

Multilingue

10+ langues, verrouillé par le glossaire

*J'ai créé **Buglt** parce qu'après des années à rédiger et relire des rapports de bug chaque jour, j'en avais assez de passer mon temps sur une mise en forme répétitive plutôt qu'à trouver de vrais problèmes. Buglt est l'outil que j'aurais aimé avoir.*

Langues : en, ja, fr, de, es, pt-br, it, ko, zh, ru, ar & plus · Solo \$39.99 (puis \$59.99) · Team \$199 (puis \$249.99) · achat unique, 5 postes par licence Équipe

Buglt · mises à jour gratuites au sein de votre version majeure tant que votre licence est active · Solo pour un utilisateur, Team jusqu'à cinq membres (aucune revente ni redistribution)

© 2026 Taskivator. All Rights Reserved. · bugit.dev